AgencyFlow

Client, Lead & Project Management Platform
for Small Digital Agencies

PROJECT DOCUMENTATION

LIVE APP	https://agencyflow-neon.vercel.app
API	https://agencyflow-api-c2kp.onrender.com
SOURCE CODE	https://github.com/shoaib1920/agencyflow
STACK	MongoDB · Express · React · Node.js · TypeScript
AUTHOR	Shoaib Liaqat
DATE	November 2026

Demo login: admin@agencyflow.dev / Admin@123 (one-click buttons on the login page)

Contents

1	Project Overview
2	Technology Stack
3	System Architecture
4	User Roles & Permissions
5	Application Pages
6	Core Features in Depth
7	Data Model
8	API Reference
9	Security
10	Testing & Code Quality
11	Deployment
12	Key Engineering Decisions
13	Running Locally

1. Project Overview

AgencyFlow is a full-stack web application that gives a small digital agency (a team that builds websites, apps or marketing for clients) one workspace for its entire business cycle: **finding work** (leads), **keeping customers** (clients) and **delivering work** (projects, tasks and files).

The problem

Small agencies typically juggle spreadsheets for leads, a separate tool for tasks, email threads for files and nothing at all for reporting. Information gets lost between tools, nobody knows the value of the sales pipeline, and team members find out about new assignments late.

The solution

- A **visual sales pipeline** (Kanban board) where deals are dragged between stages; winning a deal turns it into a client automatically.
- A **client directory** with search, filters and each client's project history.
- **Projects with tasks and file attachments**, visible only to the people working on them.
- **Real-time notifications** when a task is assigned or a deal moves.
- A **dashboard** with live KPIs and charts computed directly in the database.
- **Role-based access** (Admin, Manager, Member) enforced by the server.

Highlights at a glance

Metric	Value
Application pages	9 (Login, Register, Dashboard, Clients, Leads, Projects, Project Detail, Team, Settings)
REST API endpoints	30+ under /api
Database collections	8 (users, refreshtokens, clients, leads, projects, tasks, notifications, activities)
Automated tests	17 Jest + Supertest API tests, all passing
Language	TypeScript in strict mode on both frontend and backend
Hosting	Vercel (frontend), Render (API), MongoDB Atlas (database)

2. Technology Stack

Frontend

Technology	Purpose
React 18 + Vite	UI library and a very fast development server / bundler
TypeScript (strict)	Static types catch bugs at compile time
React Router v6	Page navigation, protected routes and role-based routes
TanStack Query	Fetching, caching and refreshing server data; optimistic updates
Zustand	Small global store for the logged-in user, theme and sidebar state
React Hook Form + Zod	Forms with schema-based validation and clear error messages
Tailwind CSS + shadcn/ui (Radix)	Styling and accessible components (dialogs, drawers, menus)
@dnd-kit	Drag and drop for the Kanban board (mouse, touch and keyboard)
Recharts	Bar, line and donut charts on the dashboard
Axios	HTTP client with an interceptor that refreshes expired tokens
socket.io-client	Receives live notifications and board updates
Sonner, Lucide, date-fns	Toast messages, icons and date formatting

Backend

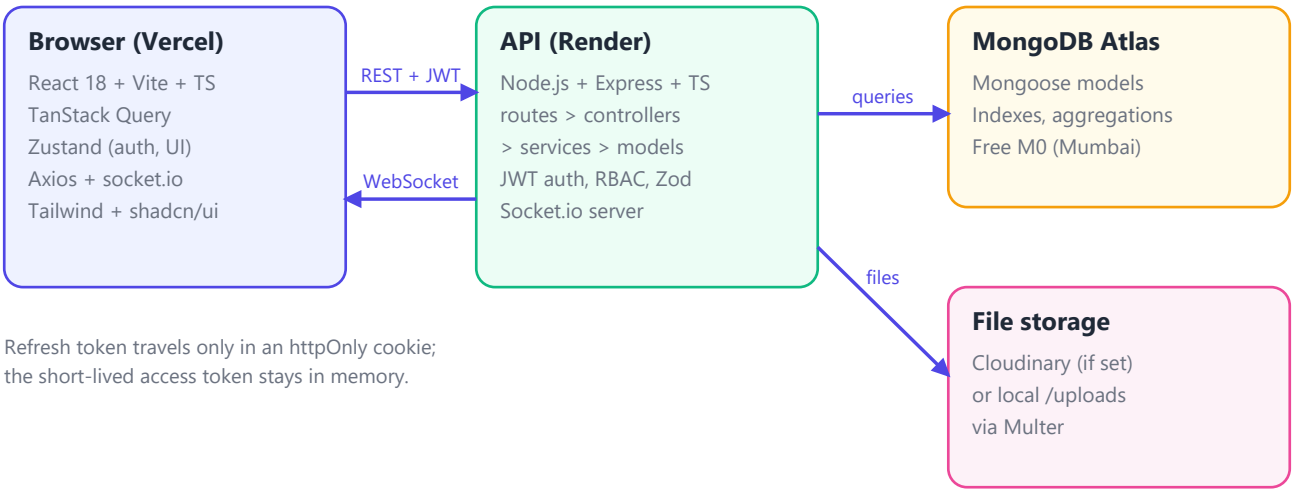
Technology	Purpose
Node.js + Express	HTTP server and routing
TypeScript (strict)	Typed controllers, services and models
MongoDB + Mongoose	Document database and schema/ODM layer
JSON Web Tokens	Short-lived access tokens and rotating refresh tokens
bcrypt	One-way password hashing (cost factor 12)
Zod	Validation of environment variables, request bodies, query strings and params
Socket.io	Real-time, authenticated WebSocket channel
Multer + Cloudinary	File uploads with a local-disk fallback
helmet, cors, express-rate-limit	Security headers, cross-origin policy, brute-force protection
morgan	HTTP request logging

Tooling & infrastructure

Tool	Purpose
Jest + Supertest + mongodb-memory-server	API tests against a throwaway in-memory database
ESLint + Prettier	Linting and consistent formatting
concurrently	One command (npm run dev) starts API and frontend together
Vercel / Render / MongoDB Atlas	Hosting for frontend, API and database

3. System Architecture

The app is split into two independent projects in one repository (a monorepo): the **client** (React single-page app) and the **server** (Express REST API + Socket.io). They communicate over HTTPS and WebSockets and are deployed separately.



Refresh token travels only in an httpOnly cookie; the short-lived access token stays in memory.

Figure 1. High-level architecture and data flow.

Backend layers

Every request passes through the same pipeline, which keeps each file small and single-purpose:

Layer	Responsibility	Example
Middleware	Security headers, CORS, rate limiting, authentication, role checks, Zod validation	`authenticate, requireRole("admin")`
Routes	Map URL + method to a controller; declare who may call it	`PATCH /api/leads/:id/stage`
Controllers	Read the HTTP request, call a service, send the response	`lead.controller.ts`
Services	Business rules: permissions scoping, conversions, notifications, activity logging	`lead.service.ts`
Models	Mongoose schemas, indexes, password hashing hooks	`Lead.ts`
Error handler	Turns any thrown error into the standard JSON envelope	`AppError`

Consistent response envelope
Every response has the same shape: `{ success, data, message, meta }`. Lists put pagination in `meta` (page, limit, total, totalPages) and validation errors put field messages in `meta.errors`.

Example request lifecycle: dragging a lead to "Won"

Step	What happens
1. Browser	The card moves instantly (optimistic update). The previous board state is kept for rollback.
2. Axios	Sends PATCH /api/leads/:id/stage with { stage: "won", position } and the access token.
3. Middleware	authenticate verifies the JWT; validate checks the body with Zod.
4. Service	Updates the lead, sets wonAt, creates a Client (only once), logs activity, stores notifications.
5. Socket.io	Pushes the notification to each manager's personal room and tells all boards to refresh.
6. Browser	Shows "Deal won! ... is now a client" with a link. If the request failed, the card snaps back.

Repository structure

Folder	Contents
server/src/config	Environment validation (Zod) and database connection
server/src/middleware	auth, requireRole, validate, upload, errorHandler
server/src/models	User, RefreshToken, Client, Lead, Project, Task, Notification, Activity
server/src/services	Business logic for each feature, plus dashboard aggregations
server/src/socket	Socket.io server, handshake authentication and rooms
server/tests	Jest + Supertest suites
client/src/pages	One component per route (lazy-loaded)
client/src/features	Feature components: client drawer, lead card, task dialog, files panel...
client/src/components	ui/ (shadcn-style primitives), layout/ (sidebar, top bar, bell), shared/
client/src/lib, stores, hooks	Axios instance, query keys, types, Zustand stores, useSocket

4. User Roles & Permissions

AgencyFlow has three roles. The **very first account** registered becomes the Admin; later sign-ups become Members, and an Admin can promote them. Permissions are enforced on the **server** by a role-guard middleware; the frontend additionally hides buttons a user cannot use, but hiding is only a convenience, never the security boundary.

Who is who

Role	Typical person	Summary
Admin	Agency owner / founder	Full control, including team management and deleting clients
Manager	Account or project manager	Runs clients and projects; can delete leads and tasks
Member	Developer, designer	Works leads and tasks; sees only the projects they are assigned to

Permission matrix

Action	Admin	Manager	Member
View dashboard, clients, leads	Yes	Yes	Yes
Create / edit clients	Yes	Yes	No (403)
Delete clients	Yes	No (403)	No (403)
Create, edit and move leads	Yes	Yes	Yes
Delete leads	Yes	Yes	No (403)
View projects	All	All	Only assigned
Create / edit / delete projects	Yes	Yes	No (403)
Create and update tasks, upload files	Yes	Yes	On own projects
Delete tasks	Yes	Yes	No (403)
Team page: add members, change roles, deactivate	Yes	No	No

> **Safety rules**
> An admin cannot demote or deactivate themselves, and the system refuses to remove the last active admin. Deactivating a user immediately revokes all of their refresh tokens, so they are signed out everywhere.

Demo accounts (seeded)

Role	Email	Password
Admin	admin@agencyflow.dev	Admin@123
Manager	manager@agencyflow.dev	Manager@123
Member	sam@agencyflow.dev	Member@123
Member	lina@agencyflow.dev	Member@123

5. Application Pages

Every signed-in page shares one layout: a **collapsible sidebar** (it becomes a slide-out sheet on phones), and a **top bar** with client search, a theme switcher (light / dark / system), the **notification bell** and the user menu. Pages are lazy-loaded, show skeleton placeholders while loading, friendly empty states when there is no data, and an error boundary instead of a blank screen if something breaks.

5.1 Login

Route: `/login`	**Visible to:** Signed-out visitors

The entry point. Users sign in with email and password. Three demo buttons fill in the seeded Admin, Manager and Member credentials so reviewers can try every role in one click.

What you can do

- Sign in with email and password (validated as you type).
- Fill demo credentials with the Admin / Manager / Member buttons.
- Go to registration.

How it works under the hood

- On success the API returns a short-lived **access token** (kept in memory) and sets a 7-day **refresh token** in an httpOnly cookie.
- Wrong email and wrong password return the same message, so the API never reveals which accounts exist.
- Login attempts are rate-limited to slow down password guessing.
- On every page load the app silently calls /auth/refresh to restore the session without asking for the password again.

5.2 Register

Route: `/register`	**Visible to:** Signed-out visitors

Creates a new account. The first account in a fresh workspace becomes the Admin; later accounts become Members.

What you can do

- Create an account with name, email, password and confirmation.
- Password rules: at least 8 characters including a letter and a number.

How it works under the hood

- The same Zod rules run in the browser (instant feedback) and on the server (the real check).
- Passwords are hashed with bcrypt before saving; the hash is never returned by the API.

5.3 Dashboard

Route: /	**Visible to:** All roles

The home page: a live summary of the whole agency.

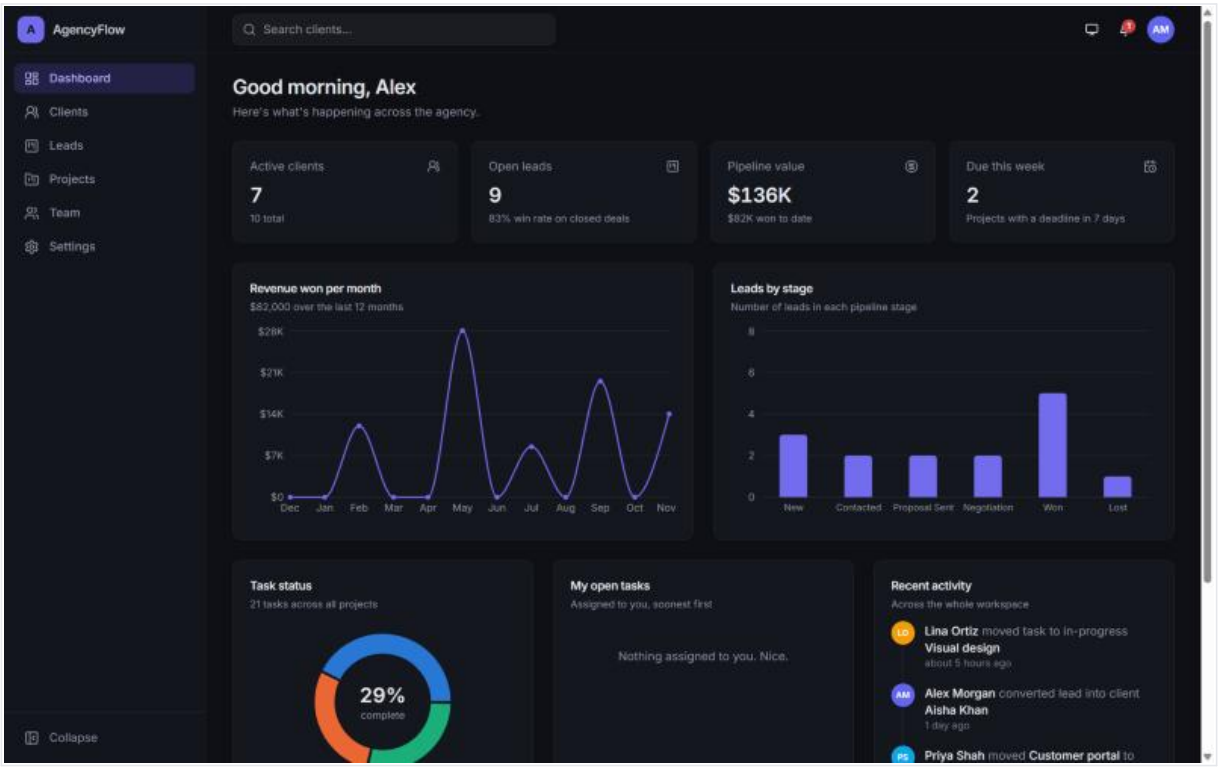


Figure 2. Dashboard (dark theme) with KPIs, revenue line chart and leads-by-stage chart.

What you can do

- **KPI cards** (each links to its page): active clients, open leads with win rate, pipeline value with revenue won, projects due in the next 7 days.
- **Revenue won per month** line chart for the last 12 months.
- **Leads by stage** bar chart.
- **Task status** donut chart with completion percentage and legend.
- **My open tasks**: tasks assigned to you, soonest due first.
- **Recent activity** timeline for the whole workspace.

How it works under the hood

- All numbers come from **MongoDB aggregation pipelines** ($match, $group, $sort) that run in parallel, not from JavaScript loops.
- Months with no revenue are filled with zero so the chart has a continuous time axis.
- The dashboard refreshes automatically when a lead moves or a notification arrives.

5.4 Clients

Route: `/clients`	**Visible to:** All roles (create/edit: Admin, Manager; delete: Admin)

A searchable directory of every client. Clicking a row opens a **detail drawer** from the right.

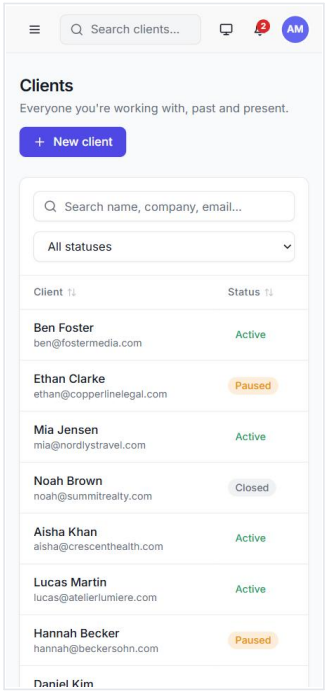


Figure 3. Clients page on a phone (light theme).

What you can do

- Search by name, company, email or country.
- Filter by status (active, paused, closed) and sort by any column.
- Page through results 10 at a time.
- Create and edit clients in a dialog form.
- Open the drawer to see contact details, notes, the client's projects and whether it came from a won lead.
- Delete a client (Admin only; blocked while the client still has projects).

How it works under the hood

- Search, filtering, sorting and pagination happen **on the server**, so the page stays fast with thousands of clients.
- List queries use `.lean()` and indexes on status and creation date.
- Filters and the open drawer are stored in the URL, so a filtered view can be bookmarked or shared.
- The top-bar search jumps here with the search already applied.

5.5 Leads (Kanban pipeline)

Route: `/leads`	**Visible to:** All roles (delete: Admin, Manager)

A drag-and-drop board of every potential deal, organised in six stages: New, Contacted, Proposal Sent, Negotiation, Won and Lost. Each column shows its card count and total deal value.

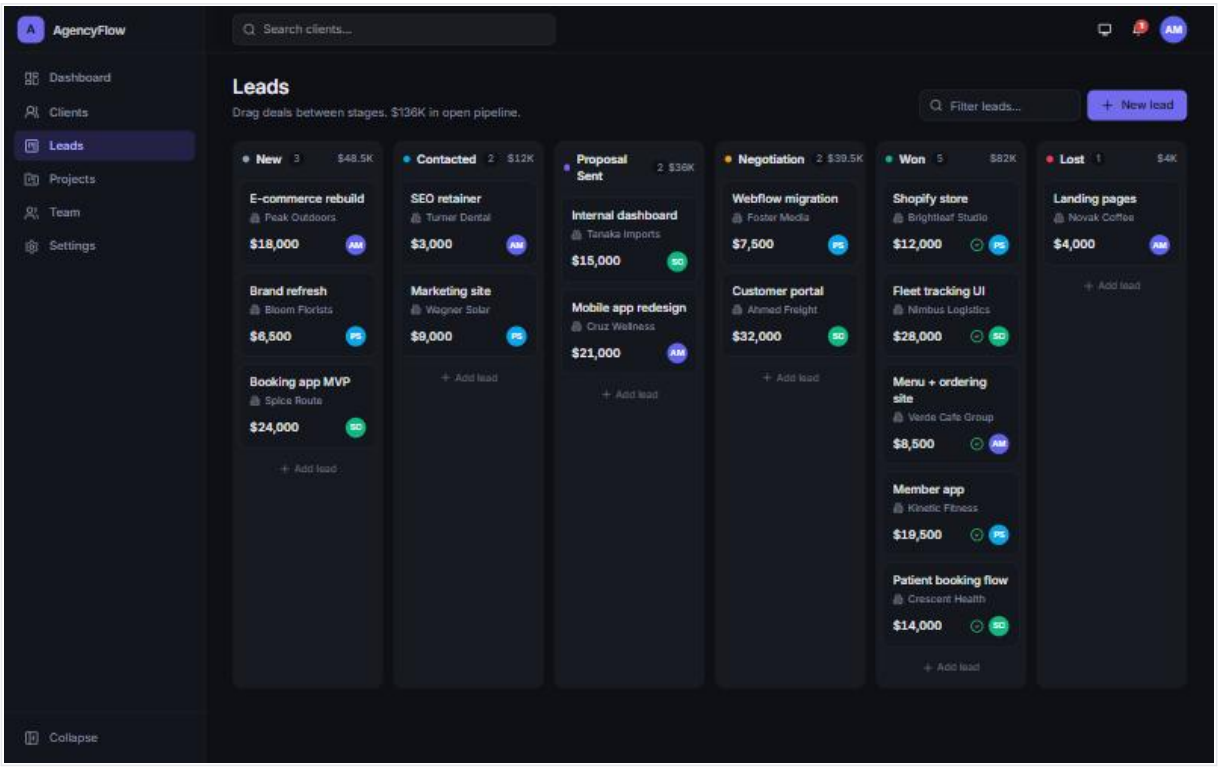


Figure 4. Leads Kanban board with stage totals and owner avatars.

What you can do

- Drag cards between and within columns (mouse, touch or keyboard).
- Add a lead directly into any column, or edit a lead by clicking its card.
- Assign an owner, deal value, source and notes.
- Filter the board by title, company or contact name.
- **Drop a card on Won**: the lead becomes a Client automatically and a toast offers "View client".

How it works under the hood

- **Optimistic update**: the board changes instantly; the server call happens in the background and the card snaps back if it fails.
- **Fractional positions**: a dropped card gets the midpoint between its neighbours (e.g. 1500 between 1000 and 2000), so only one document is written.
- Conversion to a client is **idempotent**: moving a lead out of Won and back again never creates a duplicate client.
- Managers and the lead owner get a real-time notification; every open board refreshes via a Socket.io broadcast.

5.6 Projects

Route: `/projects` **Visible to:** All roles (members see only projects they are on)

A card grid of delivery projects, sorted by nearest deadline.

What you can do

- See each project's client, status, budget, team avatars and task progress bar.
- Deadline warnings: "4d left" in amber, "overdue" in red.
- Search by title and filter by status (planning, active, on hold, completed).
- Create a project (Admin, Manager): choose the client, budget, deadline and team members.

How it works under the hood

- Task progress for all visible projects is calculated with **one aggregation** instead of one query per project.
- The server automatically narrows the query to the member's own projects, so other projects cannot be listed or opened.

5.7 Project Detail

Route: `/projects/:id` **Visible to:** Admins, Managers and project members

Everything about one project on a single page: summary cards, tasks, files and a project-specific activity timeline.

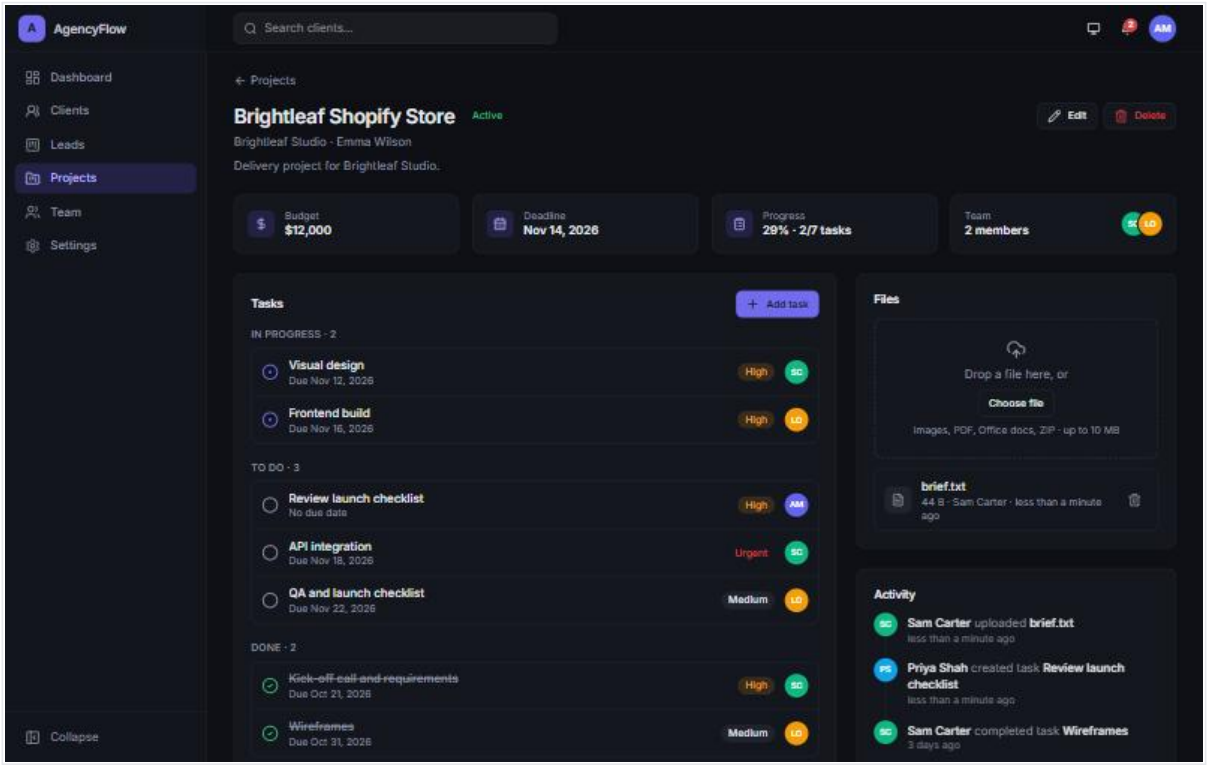


Figure 5. Project detail with grouped tasks, files and activity timeline.

What you can do

- Summary: budget, deadline, progress (done / total tasks) and team.
- Tasks grouped into In progress, To do and Done, each with priority, due date (overdue in red) and assignee.
- **One-click status cycling**: click the circle to move a task todo, in-progress, done.
- Add and edit tasks in a dialog; assigning someone sends them a live notification.
- Upload files by drag and drop or a file picker (images, PDF, Office, ZIP, up to 10 MB); download or remove them.

- Edit or delete the project (Admin, Manager). Deleting cascades to its tasks, files and activity.

How it works under the hood

- Task status changes are optimistic, like the Kanban board.
- Assigning a task to someone who is not on the project adds them to the team automatically, so they can open it.
- Uploads go to Cloudinary when configured, otherwise to the server's /uploads folder; MIME type and size are checked by Multer.

5.8 Team

Route: `/team`	**Visible to:** Admin only (the route and the sidebar link are hidden for others)

Manage who has access and what they can do.

What you can do

- Add a team member with a temporary password and a role.
- Change any other member's role from a dropdown.
- Deactivate or reactivate accounts.
- See each person's title, email and last sign-in time; read a summary of what each role can do.

How it works under the hood

- Protected twice: the frontend route requires the admin role and every /api/users write is guarded by `requireRole("admin")`.
- Role changes are recorded in the activity log.

5.9 Settings

Route: `/settings`	**Visible to:** All roles

Personal profile, appearance and security.

What you can do

- Edit name, job title and avatar colour.
- Choose light, dark or system theme (remembered on this device).
- Change password.

How it works under the hood

- Changing the password revokes every other refresh token, signing the user out on other devices.
- The theme is applied before the first paint to avoid a white flash in dark mode.

5.10 Shared layout: sidebar, top bar and notifications

- **Sidebar**: Dashboard, Clients, Leads, Projects, Team (admins only) and Settings; collapses to icons on desktop and turns into a slide-out sheet on mobile.
- **Top-bar search**: jumps to the Clients page with the search applied.
- **Notification bell**: unread badge, dropdown of the latest 20 notifications, "mark all read", and clicking an item opens the related page.
- **User menu**: name, email, role badge, Settings and Sign out (sign-out revokes the refresh token on the server).

6. Core Features in Depth

6.1 Authentication with refresh-token rotation

AgencyFlow uses two tokens so that a stolen credential is either short-lived or quickly detected:

Token	Lifetime	Stored in	Why
Access token (JWT)	15 minutes	Browser memory only	Sent on every API call; expires quickly if leaked
Refresh token (JWT)	7 days	httpOnly, Secure cookie	JavaScript cannot read it, so XSS cannot steal it

- **Rotation**: every refresh returns a brand-new refresh token and retires the old one.
- All tokens from one login share a **family** ID. Only a hash of each token is stored in MongoDB.
- **Reuse detection**: if an already-retired token is presented again, someone must have copied it, so the server revokes the **entire family** and every session from that login must sign in again.
- The frontend Axios interceptor catches a 401, refreshes once and retries the original request. Concurrent 401s share a single refresh call so they do not trigger reuse detection by accident.
- Expired tokens are removed automatically by a MongoDB TTL index.

6.2 Real-time notifications (Socket.io)

- The socket authenticates with the same access token during the handshake.
- On connect, each socket joins **user:<id>** and **role:<role>** rooms. Emitting to user:<id> reaches every tab and device of that person.
- Notifications are saved in MongoDB first, then pushed live, so offline users still see them in the bell later.
- Events: `notification:new` (to one user) and `leads:changed` (broadcast, refreshes all boards).
- Triggers: a task is assigned to you; a lead changes stage (owner and managers are notified); a deal is won. The person who caused the event is never notified about their own action.

6.3 Dashboard aggregations

Metric	Pipeline
Active / total clients	$group with a conditional $sum on status
Leads by stage + pipeline value	$group by stage: count and $sum of value
Revenue won per month	$match stage = won and wonAt in last 12 months, $group by year+month, $sort
Task status	$group by status
Projects due this week	$match deadline in next 7 days and not completed, $count

6.4 Activity log

Services record who did what (created a client, moved a lead from Proposal Sent to Negotiation, completed a task, uploaded a file, changed a role). Entries linked to a project also power that project's own timeline. Logging is fire-and-forget: a logging failure never breaks the user's action.

6.5 File uploads

- Multer keeps the file in memory, validates type (images, PDF, text, Office, ZIP) and size (10 MB).
- With Cloudinary keys the file is streamed to Cloudinary; without them it is written to /uploads and served statically.
- Filenames are sanitised and prefixed with a timestamp and random bytes.

7. Data Model

Eight MongoDB collections, defined with Mongoose schemas. Indexes are placed on the fields used for filtering and sorting.

Collection	Key fields	Indexes / notes
users	name, email (unique), password (hashed, hidden), role, title, avatarColor, isActive, lastLoginAt	role; password excluded from JSON

Collection	Key fields	Indexes / notes
refreshtokens	user, jti (unique), family, tokenHash, revokedAt, replacedBy, expiresAt	TTL index on expiresAt
clients	name, company, email, phone, country, status, notes, sourceLead, createdBy	{status, createdAt}, text index
leads	title, contactName, company, email, value, stage, position, owner, client, wonAt	{stage, position}, {stage, wonAt}, owner
projects	title, description, client, budget, deadline, status, members[], attachments[]	client, members, {status, deadline}
tasks	project, title, assignee, priority, status, dueDate, completedAt	{project, status}, {assignee, status}
notifications	recipient, actor, type, title, body, link, read	{recipient, read, createdAt}
activities	actor, action, entityType, entityId, entityName, project, meta	createdAt, {project, createdAt}

Relationships

- A **Client** has many **Projects**; a **Project** has many **Tasks** and embedded **attachments**.
- A **Lead** that is won points to the **Client** it created, and the client points back via sourceLead.
- Users are referenced as lead owners, project members, task assignees, notification recipients and activity actors.

8. API Reference

Base URL: `https://agencyflow-api-c2kp.onrender.com/api`. Authenticated endpoints need `Authorization: Bearer <accessToken>`.

Method	Endpoint	Access	Description
GET	/health	Public	Service and database status
POST	/auth/register	Public	Create account (first user becomes admin)
POST	/auth/login	Public	Access token + refresh cookie
POST	/auth/refresh	Cookie	Rotate refresh token, new access token
POST	/auth/logout	Cookie	Revoke the session family
GET / PATCH	/auth/me	Auth	Current user / update profile
PATCH	/auth/me/password	Auth	Change password, sign out other devices
GET	/users	Auth	Team list
POST / PATCH	/users, /users/:id	Admin	Add member / change role or status
GET	/clients	Auth	page, limit, search, status, sort, order
GET	/clients/:id	Auth	Client with its projects
POST / PATCH	/clients, /clients/:id	Admin, Manager	Create / update
DELETE	/clients/:id	Admin	Delete (blocked if it has projects)

Method	Endpoint	Access	Description
GET / POST	/leads	Auth	Board data / create lead
PATCH	/leads/:id	Auth	Update lead
PATCH	/leads/:id/stage	Auth	Move lead; Won creates a client
DELETE	/leads/:id	Admin, Manager	Delete lead
GET	/projects, /projects/:id	Scoped	Members only see their projects
POST / PATCH / DELETE	/projects[/:id]	Admin, Manager	Manage projects
GET	/projects/:id/activity	Project access	Project timeline
POST / DELETE	/projects/:id/files[/:fileId]	Project access	Upload (field "file") / remove
GET / POST	/projects/:projectId/tasks	Project access	List / create tasks
PATCH	/projects/:projectId/tasks/:taskId	Project access	Update task (assigning notifies)
DELETE	/projects/:projectId/tasks/:taskId	Admin, Manager	Delete task
GET	/dashboard	Auth	KPIs, chart data, activity, my tasks
GET	/activity	Auth	Workspace activity feed
GET	/notifications	Auth	Latest 20 plus unread count
PATCH	/notifications/:id/read, /read-all	Auth	Mark as read

Error handling
Errors use the same envelope with `success: false`. Status codes: 400 validation, 401 not signed in or token expired, 403 role not allowed, 404 not found (also used for projects a member may not see), 409 conflict (duplicate email, client with projects).

9. Security

Area	Measure
Passwords	bcrypt hashing (cost 12); never returned by the API; strength rules on both client and server
Sessions	In-memory access token, httpOnly refresh cookie, rotation with reuse detection, revocation on logout, password change and deactivation
Authorisation	requireRole middleware on every protected write; members' project queries are scoped on the server
Input validation	Zod schemas for every body, query and param; invalid ObjectIds rejected; search input regex-escaped
HTTP hardening	helmet security headers, CORS restricted to the frontend origin, 1 MB JSON limit
Brute force	express-rate-limit on /api/auth (100 requests per 15 minutes)
Cross-domain cookies	In production: SameSite=None; Secure; HttpOnly, path limited to /api/auth, Express trusts the proxy
Uploads	MIME allow-list, 10 MB limit, sanitised filenames
Secrets	Loaded from environment variables validated at startup; .env files are git-ignored
Information leaks	Identical message for unknown email and wrong password; stack traces hidden in production

10. Testing & Code Quality

The API has **17 automated tests** written with Jest and Supertest. Each run starts a fresh in-memory MongoDB, so tests never touch real data.

Suite	What is verified
Auth (6)	First user is admin; duplicate email rejected; validation errors; login success and failure; /me; refresh rotation and reuse detection revoking the whole family
Role guard (4)	Member cannot create clients; manager cannot delete clients; only admins manage the team; tampered tokens rejected
Clients CRUD (4)	Create, read, update, delete; validation; server-side pagination, search, filter and sort; malformed IDs
Lead stage change (3)	Move and position update; activity logged; managers notified; Won converts to exactly one client; invalid stage rejected

- TypeScript **strict mode** on both sides with no unjustified `any`.
- **ESLint** (typescript-eslint, react-hooks) and **Prettier** configured for the whole repository.
- Every feature was also checked manually in the browser: login, dashboard, drag-to-Won conversion, live notifications, uploads, mobile layout and light/dark themes.

11. Deployment

Part	Platform	URL / notes
Frontend	Vercel	agencyflow-neon.vercel.app. Root directory "client"; vercel.json rewrites every route to index.html (SPA)
API	Render (free web service, Singapore)	agencyflow-api-c2kp.onrender.com. Defined in render.yaml; health check on /api/health
Database	MongoDB Atlas (free M0, Mumbai)	Seeded once from a local machine with npm run seed
Source	GitHub	github.com/shoaib1920/agencyflow; Render and Vercel redeploy automatically on push to main

Environment variables

Variable	Where	Purpose
MONGO_URI	Render	Atlas connection string including the /agencyflow database name
CLIENT_URL	Render	Allowed browser origin for CORS (the Vercel URL)
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	Render	Signing keys, generated by Render
NODE_ENV=production	Render	Enables secure cross-site cookies and hides stack traces
CLOUDINARY_*	Render (optional)	Durable file storage; without it uploads are lost on redeploy
VITE_API_URL	Vercel	Base URL of the API used by the frontend and the socket

Issues solved during deployment

Symptom	Cause	Fix
Render crashed with ReplicaSetNoPrimary	Atlas only allowed the developer's own IP address	Allowed access from anywhere (0.0.0.0/0) in Atlas Network Access
API live but demo login failed	Connection string had no database name, so the app used the empty "test" database	Added /agencyflow to MONGO_URI
Browser blocked requests (CORS)	CLIENT_URL did not match the Vercel origin	Set CLIENT_URL to the exact Vercel URL
API started on the wrong port locally	A dev tool injected PORT for the frontend	API reads API_PORT before PORT

> **Free-tier note**
> The free Render instance sleeps after about 15 minutes without traffic, so the first request after a pause can take 30 to 50 seconds. Opening the site a few minutes before a demo avoids the wait.

12. Key Engineering Decisions

Decision	Reasoning
Layered backend (routes, controllers, services, models)	Each file has one job. Business rules live in services that tests and the seed script can reuse without HTTP.
TanStack Query for server data, Zustand for UI state	Caching, background refresh and optimistic updates come built in; the global store stays tiny.
Optimistic Kanban with fractional positions	Instant feedback for users and one database write per drop.
Aggregation pipelines for statistics	The database does the counting next to its indexes and returns a handful of rows.
Socket.io rooms per user	One emit reaches every tab of the right person without tracking socket IDs.
Refresh-token rotation with reuse detection	Industry-standard protection against stolen long-lived tokens.
Zod on both sides	One validation language for env, requests and forms; parsed data arrives typed.
Separate hosts for frontend and API	Vercel serverless cannot hold WebSocket connections, so the API runs on a persistent Render server.

13. Running Locally

Requirements: Node.js 20+ and MongoDB (local or Atlas).

Step	Command
Install everything	`npm run install:all`
Create the API config	`cp server/.env.example server/.env`
Load demo data	`npm run seed`
Start API (:5000) and web (:5173)	`npm run dev`
Run the test suite	`npm test`
Lint / format	`npm run lint` / `npm run format`

Built in November 2026 as a portfolio project. Source code: github.com/shoaib1920/agencyflow